

交付包独立复现测试报告

2026 高教社杯全国大学生数学建模竞赛 A 题《药材的烘干问题》
只使用最终交付文件夹里的两个文件，在全新目录中完整跑一遍

被测文件	最终交付/支撑材料.zip (5,849,678 B, 127 个条目) 最终交付/论文全文_交付版.pdf (1,436,931 B, 83 页)
测试方式	在全新目录中解压、校验、按包内 README 与 交付说明的步骤完整运行；全程不修改任何代码，只新增测试记录
环境	macOS (Apple Silicon) 、Python 3.9.6 (Xcode Command Line Tools) 、新建独立 venv；依赖严格按 requirements-lock.txt 安装：numpy 2.0.2 / openpyxl 3.1.5 / matplotlib 3.9.4 / seaborn 0.13.2。另装 PyMuPDF 1.26.5，仅用于验证 make_delivery_pdf.py
耗时	全量复现 ./reproduce.sh：59 分 48 秒（文档预计约 64 分钟）；轻量复现 ./reproduce.sh --fast：8 分 40 秒（文档预计约 8 分钟）
总结论	论文 PDF 与四个结果文件完全可复现；但发现 2 处必须处理的问题（论文 § 11 收缩判据的数字无法用包内脚本复现；随包的数据自检报告是旧版本、与论文自相矛盾），另有 6 项中等或轻微问题

一、结论速览

检查项	结果	说明
解压与文件完整性	通过	127 个条目，中文名正常，UTF-8 标志位已设置
setup.sh 建环境 + check_env	通过	四个依赖版本与 requirements-lock.txt 完全一致，中文字体齐全
./reproduce.sh 全量复现	通过	59 分 48 秒跑完，四个 result 文件、对照 CSV、figdata、40 张插图全部生成
四个 result*.xlsx	通过	result1/3/4 与随包文件逐格零差异；result2 全表逐格零差异（413402 行）
正文表格与对照 CSV	通过	问题一-表1表2.md、问题二三四-表格.md、两份对照 CSV 逐字节一致
插图派生数据 figdata	通过	CSV/JSON 逐字节一致，两个 npz 数值完全一致
论文 PDF 格式合规	通过	83 页 A4；页边距 2.71/2.67/3.68/2.51 cm；页码居中连续；第 1 页为摘要页；无身份信息
论文 PDF 可复现	通过	包内 tex 重新编译 + 包内交付脚本，逐页墨迹包围盒偏差 0.000 pt、83/83 页文字一致
打包脚本 ./打包支撑材料.sh	通过	默认包 6 MB；加 --with-result2 实测 16.4 MB，同样 ≤ 20 MB
随包 输出/基准值-论文表1到表6.json	过期	表 6 仍是旧值（1.4770...），与论文表 6 / result4.xlsx（1.7188...）不符
包内数据自检 verify_paper_numbers.py	不通过	共审计 365 项，10 项不一致，全部集中在论文 § 11（收缩判据与代价）
补充分析 check_consistency.py	不通过	共 57 项，8 项 FAIL（“路径”一节的体积平均含水率与偏差）
随包 输出/数据自检报告.json	过期	347 项、全部标记一致；其中问题四 50.31 h 与论文 51.08 h 冲突
随包 输出/figs 插图	部分过期	7 张与重算或论文用的版本不一致（其余 33 张仅 PDF 时间戳不同）
README 第五节自检命令（纯净解包）	无法运行	缺 输出/result*_data.npz（被打包脚本排除）
问题一表 1/表 2 的末位小数	受时间步限制	Δt 由 1 s 细到 0.0625 s：表 1 有 33/35 格、表 2 有 9/35 格末位变化（最大 1.8e-3 °C），论文却写“时间步已收敛”

二、测试环境与做法

测试严格按“只交付两个文件”的场景进行：把 支撑材料.zip 解压到一个全新的空目录，在包内建虚拟环境、按 README 的步骤复现，再用独立脚本核对结果，全程不修改包内任何文件。为了区分“环境问题”和“包本身的问题”，关键结论都在两到三个独立副本中重复验证过。

环境与依赖（实测版本）

项目	实测值
操作系统 / 架构	macOS (Apple Silicon, darwin)
Python	3.9.6 (系统自带 /usr/bin/python3, 指向 Xcode Command Line Tools)
虚拟环境	由 ./setup.sh 新建 .venv；四个依赖按 requirements-lock.txt 精确命中
依赖版本	numpy 2.0.2；openpyxl 3.1.5；matplotlib 3.9.4；seaborn 0.13.2
额外依赖	PyMuPDF 1.26.5（仅用于验证 make_delivery_pdf.py，requirements.txt 中未列）
LaTeX	环境原无 TeX；改用环境中自带的 Tectonic + 本机缓存编译包内 tex，成功

用到的命令（均为包内脚本，未做任何修改）

```
./setup.sh
./reproduce.sh # 全量复现（结果 + 对照 + 派生数据 + 插图 + 自检）
./reproduce.sh --fast # 轻量复现（四个 result 文件 + 正文表格）
./venv/bin/python 代码/verify_paper_numbers.py --json 输出/数据自检报告.json
./venv/bin/python 代码/make_delivery_pdf.py 论文/论文全文.pdf 论文/论文全文_交付版.pdf
./打包支撑材料.sh
./打包支撑材料.sh --with-result2 # 实测 16.4 MB
./venv/bin/python 代码/export_baseline_tables.py --tex 论文/论文全文.tex
./venv/bin/python 补充分析/斜率截距与收缩/check_consistency.py
```

三、通过的检查（实测数据）

3.1 交付包可解压、内容与原始工作区一致

支撑材料.zip 共 127 个条目，中文文件名正常显示；zip 条目的 UTF-8 标志位 (bit 11) 在非 ASCII 文件名上均已设置，Windows 下不会因代码页出现乱码。逐条与原始工作区文件做 MD5 比对：127/127 完全相同，说明交付包就是最终工作区状态。

3.2 一键建环境可用

./setup.sh 在全新目录中成功建立 .venv 并安装四个依赖（需联网）。代码/check_env.py 输出：numpy 2.0.2、openpyxl 3.1.5、matplotlib 3.9.4、seaborn 0.13.2，并识别到 Arial Unicode MS / Hiragino Sans GB / Songti SC / STHeiti 等中文字体，环境自检通过。安装过程中出现的 pip 缓存目录与 matplotlib 配置目录不可写告警，会自动回落到临时目录，不影响结果。

3.3 全量复现成功，用时与文档相符

./reproduce.sh 依次完成：环境自检 → 问题一求解与 result1.xlsx → 问题二/三/四求解与 result2/3/4.xlsx → 20 组网格与口径对照、三种口径对照、灵敏度、收缩判据 → 插图派生数据 → 40 张插图与版面自检 → 数据自检，全程 59 分 48 秒，无中断、无报错退出。问题三、问题四的烘干时间分别为 206700 s (57.42 h) 与 183885 s (51.08 h)，与论文一致。

3.4 数值结果与随包文件一致

对比对象	方法	结果
输出/result1.xlsx (2 张表 × 1800 行 × 22 列)	逐格比对	差异 0 格
输出/result3.xlsx (3445 行 × 22 列)	逐格比对	差异 0 格
输出/result4.xlsx (3064 行 × 22 列 + 药材表面)	逐格比对	差异 0 格
输出/result2.xlsx (206700 行数据 × 22 列 × 2 张表; 随包不含, 与工作区副本比对)	逐格全表比对 (413402 行)	差异 0 格
输出/问题一-表1表2.md、问题二三四-表格.md	逐字节	完全一致 (MD5 相同)
输出/问题三-网格与口径对照.csv、问题三-三种口径对照.csv	逐字节	完全一致
输出/figdata/* (5 个 CSV/JSON + 2 个 npz)	逐字节 / 逐数组	CSV/JSON 一致; npz 最大数值差 0
9 张论文插图的文字内容	重新生成后与 论文/figs 比对	9/9 文字完全一致

补充说明：40 张插图中，33 张的矢量内容流与随包文件逐字节相同（差异只在 PDF 的时间戳与元数据）；另 7 张内容确实不同（见 4.3）。此外，论文/figs 里那 9 张与重跑 make_figures.py 得到的插图文案一致、视觉一致，但矢量内容流并不完全相同（属渲染层差异），若要“图与脚本严格同源”，需要重跑 make_figures.py 并把结果同步回论文/figs 后再编译。

3.5 论文 PDF 的格式与“可复现性”都通过

对 论文全文_交付版.pdf 做了独立核对（不使用包内脚本）：

检查项	实测	结论
页数 / 页面尺寸	83 页；595.28 × 841.89 pt (A4)	通过
四边最小页边距（逐页量墨迹包围盒）	左 2.71 / 右 2.67 / 上 3.68 / 下 2.51 cm	均 ≥ 2.5 cm，通过
页码	页脚居中，1 至 83 连续	通过
第 1 页	标题 + 摘要 + 关键词（无承诺书、无编号页）	通过
身份信息	无学校、学院、姓名、学号、手机号、邮箱；PDF 无作者等元数据	通过
正文页数	正文第 2 至 30 页（29 页），参考文献自第 31 页起	通过
文字可用性	文字可选、可检索（可完整抽取全文）	通过

更强的验证：用包内 论文/论文全文.tex 重新编译（得 83 页），再用包内 代码/make_delivery_pdf.py 生成交付版，与随包的交付版逐页比对：每页墨迹包围盒偏差 0.000 pt，83/83 页文字一致，脚本打印的缩放比例 0.90778 与页边距 2.71/2.67/3.68/2.50 cm 也与交付说明完全吻合。即论文 PDF 可从包内源文件逐页复现。

3.6 打包脚本可用、体积合规

./打包支撑材料.sh 在复现后的目录里重新打包成功：127 个条目、6 MB（上限 20 MB），条目构成与随包一致；脚本给出的体积提示与提交前重命名提醒都正确。

另外实测了脚本的另一个用法：./打包支撑材料.sh --with-result2（把 27.3 MB 的 result2.xlsx 一并打包）生成的是 128 个条目、16.4 MB 的压缩包，脚本自检也判为“符合 ≤ 20 MB”，包内 result2.xlsx 与源文件 MD5 相同、unzip -t 无错。这一点与 README / 交付说明里“约 38 MB、会超过 20 MB 限制”的说法不一致（见 4.8）。

四、发现的问题（按优先级）

4.1 【高】论文 § 11（收缩判据与代价）的数字无法用包内脚本复现

全量复现后执行包内自检脚本，结果为“共审计 365 项，不一致 10 项”，全部落在 § 11：

组	项目	论文值	复算值
§ 11几何	ρ 反演收缩 / h	29.45	30.94
§ 11几何	实测收缩 / h	23.85	25.21
§ 11几何	ρ 反演相对固定 / %	-48.6	-46.0
§ 11几何	实测相对固定 / %	-58.4	-56.0
§ 11扫描	b=0, t_{implied} / h	21.27	22.96
§ 11扫描	b=0, Δ / %	169	150
§ 11扫描	b=128, t_{implied} / h	29.45	30.94
§ 11扫描	b=128, Δ / %	95	85
§ 11扫描	b=320, t_{implied} / h	40.59	41.63
§ 11扫描	b=320, Δ / %	41	38

现象：随包的 输出/收缩判据与代价.json（与论文表 11、附录 B.4 表 20 的数字一致）和包内代码/shrink_criterion.py 的重算结果不一致——固定半径那一列（ t_{fixed} ：57.30/57.31/57.32/57.36/57.43 h）两边完全相同，只有含收缩的一列不同，说明求解器主体没有问题，差异集中在“收缩闭合并算 / 实测半径”这一路径。

已排除随机性与并行度：把同一条命令分别以 --jobs 1、--jobs 4、--jobs 8 在三个独立副本中重跑，几何对照结果均为 fixed 57.32 h / measured 25.21 h / implied 30.94 h，完全一致，即这是确定性的口径或代码改动差异，不是并行竞争导致的抖动。

根因可以在项目仓库里查实（交付的 zip 不含 .git，只拿到压缩包的复核者看不到这段历史）：
git 历史里 commit 9829780（09-12 22:57）就是现在这份 输出/收缩判据与代价.json 的来源——把该提交里的同名文件取出，几何对照正是 fixed 206365 / measured 85845 / implied 106015 s（= 57.32 / 23.85 / 29.45 h），b 扫描首行 t_{implied} = 21.275 h，与随包文件逐位相同。而 commit 1949f82（09-13 00:53，“同步项目最新代码：问题四改为经典 Fick 口径（去掉对流项）”）删掉了 solve_p234.py 在贴体坐标下的对流项（旧代码里的 $\text{adv} = \text{dt} \cdot \text{accum} \cdot r_{\text{now}} \cdot r_{\text{dot}} \cdot \text{xi}/2$ ），收缩路径因此改变，重算得 90750 s / 111395 s。所以论文 § 11 的这几个数是 00:53 那次改动之前的旧结果，那一轮只把 result*.xlsx 与正文主体更新了，§ 11 的数字、JSON、插图等衍生件都留在了旧口径上。

另外，shrink_criterion.py 的 main() 是“先把旧 JSON 读回来、只覆盖本次命令算到的节、再整份写回”（见该文件第 186 – 188 行），所以只跑 --analytic 时几何对照/扫描两节会原样保留、而文件时间戳照样刷新——“文件很新、数字很旧”就是这么来的。建议顺手堵上：在 reproduce.sh 里先删掉旧 JSON，或给每笔记上生成时间与代码指纹。

影响面：正文表 11（§ 11.4“建模开关对烘干时长的影响”）里那两行几何对照（29.45 h / -48.6%、23.85 h / -58.4%）、§ 11.3 的时长判据阈值（0.913 / 0.835 / 0.704，按重算值应约为 0.906 / 0.823 / 0.683）、§ 11.6 第 3 条的“时长口径下 Δ 达 95%”，以及附录 B.4（收缩判据的闭式推导与参数扫描）表 20 中 b = 0、128、320 等行的“收缩时长”和 Δ 都需要复核。另外，论文第 81 页附录里嵌入的 verify_paper_numbers.py 清单把这几个值（29.45 / 23.85 / 21.27 / 169 / 95 / 41）硬编码在脚本里，也要同步修改——否则改完正文再自检必然报错。论文正文中与四个问题直接相关的结论（57.42 h、51.08 h 等）不受影响。

建议：（a）以包内脚本重算值为准，更新论文 § 11 与表 20；或（b）若论文值来自另一版口径，请在包内补上能产生该值的脚本与参数，并重跑 输出/收缩判据与代价.json，使论文、JSON、脚本三者自洽。

4.2 【高】随包的输出/数据自检报告.json 是旧版本，与论文自相矛盾

README 第五节把 输出/数据自检报告.json 作为“包内已经验证过”的证据，但该文件与论文、与随包的 result4.xlsx 都对不上：

项目	随包报告	论文 / 重算值
问题四烘干时长	181115 s = 50.31 h	183885 s = 51.08 h
result4 行数（不含表头）	3018	3064
result4 末时刻 / s	181080	183840
报告条目总数	347 项（全部标记一致）	365 项（其中 10 项不一致）

判断：这份报告是用更早一轮的结果打出来的，随包发布时没有重新生成。它恰好是“问题四烘干时间”这一核心结论在支撑材料里唯一看似冲突的地方。

建议：重新运行 代码/verify_paper_numbers.py 覆盖该文件（注意先解决 4.1 的 10 项不一致），然后重新打包。

4.3 【中】随包插图中有 7 张与重算结果或论文用图不一致

40 张插图中，33 张与重算结果只有 PDF 时间戳差异（文字与内容一致），其余 7 张内容不同：

插图	随包版本	重算 / 论文版本	性质
d29_geo_shrink.pdf	“线收缩 <3%”“中心含水率”-19.5%”	“线收缩 <5%”“体积平均含水率”-5.6%”	论文用图，随包版本过期
d19_p4_shrink.pdf、d20_p4_fiel d.pdf	与论文用图不同	与 论文/figs 一致	论文用图，随包版本过期
s07_roadmap.pdf	标注 tdry=50.31 h	tdry=51.08 h	标注旧结果，与论文冲突
d21_p4_surface.pdf	1.5 梯度最大 -2.04 (t=2.0 h)	-1.77 (t=2.8 h)	标注旧结果
d25_dry_mass.pdf	末态 -7%	末态 -4%	标注旧结果
d04_radius_vs_moisture.pdf	文字相同，曲线细节略有差异	—	轻微

其中 s07_roadmap.pdf 直接把问题四的烘干时间写成 50.31 h，与论文的 51.08 h 相冲突；d19 / d20 / d29 三张是论文正文插图，随包副本用的是更早版本。这些问题在重跑 make_figures.py 后自动消失。

4.4 【中】补充分析一节的一致性自检有 8 项失败

补充分析/斜率截距与收缩/check_consistency.py 用来核对该节 .tex / README 里的数字，实测“共 57 项，失败 8 项”，集中在“路径”一节：5 h / 10 h / 20 h / 50 h 的体积平均含水率与偏差，例如 5 h 平均 C 正文 0.959、计算值 1.205；10 h 正文 0.510、计算值 0.621；偏差范围上界正文 22.4%、计算值 28.4%。

说明：该节其余 49 项（判据、阈值、收缩系数、总量互证等）全部通过，问题集中在这一小节的数字没有随最后一次重算同步。建议重跑 compute_shrinkage.py 后修正该节文字。

4.5 【中】纯净解包状态下，文档里的部分自检命令跑不起来

文档位置	命令	实测结果
README 第五节	./venv/bin/python 代码/verify_paper_numbers.py	FileNotFoundError：输出/result23_data.npz（result1_data.npz、result4_data.npz 同样缺失）
补充分析/…/README.md	./venv/bin/python compute_shrinkage.py	FileNotFoundError：输出/result4_data.npz

文档位置	命令	实测结果
交付说明 第二节	python3 代码/make_delivery_pdf.py 论文/论文全文.pdf ...	先报 ModuleNotFoundError: fitz (PyMuPDF 不在 requirements.txt) ; 补装后仍找不到论文/论文全文.pdf (该文件按设计未打进包)

三个 *_data.npz 属于“.gitignore 里可以由脚本重算的大文件”，打包脚本也排除了它们，所以必须先把代码/solve_p1.py 与 代码/run_all.py 跑一遍才能自检；README 里没有写明这一前置步骤，审阅者按文档直接执行会误以为包损坏。make_delivery_pdf.py 则需要额外的 PyMuPDF 与一个 LaTeX 引擎，都属于“包外依赖”。

失败顺序也实测过：先在 输出/result23_data.npz 上崩（不是 result2.xlsx）；把三个 *_data.npz 补进包里再跑，才会轮到 输出/result2.xlsx (openpyxl 报 FileNotFoundError)。所以只给读取 xlsx 的那一步加 try/except 并不能解决问题，必须在 README 里写明前置步骤，或让自检脚本在缺文件时优雅降级并打印“缺 X，跳过 N 项”。

4.6 【中】问题一表 1 / 表 2 的末位小数受时间步限制，而论文称“时间步已收敛”

论文第 11 页写“网格与时间步均已收敛”，并在表 1 / 表 2 里按“保留四位小数”给出数值，但把包内求解器的时间步细化后，末位数字会变：

算例 (N=400, 物理时长 1800 s)	$\Delta t=1\text{ s}$ (交付)	$\Delta t=0.25\text{ s}$	$\Delta t=0.125\text{ s}$	$\Delta t=0.0625\text{ s}$
表 1: $t=600\text{ s}$, 中心温度 / $^{\circ}\text{C}$	28.4549	28.4537	28.4535	28.4534
表 1: $t=1800\text{ s}$, 中心温度 / $^{\circ}\text{C}$	33.5765	33.5756	33.5755	33.5754
表 2: $t=100\text{ s}$, 表面含水率 / (kg/kg)	2.2475	2.2472	2.2472	2.2471

逐格统计： $\Delta t=1\text{ s}$ 与 $\Delta t=0.0625\text{ s}$ 相比，表 1 有 33/35 格、表 2 有 9/35 格（7 个表面格 + 2 个 1.5 cm 格）末位不同，最大偏差 $1.8\text{e-}3\text{ }^{\circ}\text{C}$ （出现在 $t=900\text{ s}$ 的中心节点）（ $\Delta t=1\text{ s}$ 的后向 Euler 在时间方向是一阶，误差 $\propto \Delta t$ ）。也就是说，“时间步已收敛”这句话在四位小数的意义下并不成立，交付的表 1 / 表 2 末位是 $\Delta t=1\text{ s}$ 的离散误差。

两点补充：其一，这个量级 ($1.5\text{e-}3\text{ }^{\circ}\text{C}$) 远小于论文自己给出的 $0.02\text{ }^{\circ}\text{C}$ 边界条件噪声（第 27 页第 9 条），而该条已经说明“温度结果只有 3 位有效数字”——所以问题不在物理结论，而在“四位小数 + 时间步已收敛”这个表述与数据的自治性；其二，把 Δt 改成 0.25 s 只多花约 30 秒（本次实测 27 秒），是本报告里性价比最高的一处修改。

收敛速度的定量补充（这一步很关键）：把 Δt 细化到 0.25 s 并不能让四位小数落定—— $\Delta t=0.25\text{ s}$ 与 0.0625 s 相比，表 1 仍有 32/35 格末位不同（最大偏差 $4.0\text{e-}4\text{ }^{\circ}\text{C}$ ）；继续细到 0.03125 s ，表 1 仍有 7/35 格末位不同（最大 $1.0\text{e-}4\text{ }^{\circ}\text{C}$ ，此时表 2 已完全一致）。按一阶收敛估算，要让表 1 的四位小数全部落定需要 $\Delta t \approx 0.01\text{ s}$ 量级（ $N=400$ 下约 10 分钟），而 $\Delta t=0.25\text{ s}$ 只多花约 30 秒、只把误差从 $1.8\text{e-}3$ 降到 $4\text{e-}4$ 。

因此建议二选一（不要用 $\Delta t=0.25\text{ s}$ 交差，也不要再在论文里写“离散误差 $< 5\text{e-}5$ ”——那个量级在 $\Delta t=0.25\text{ s}$ 下并不成立）：(a) 把第 11 页“网格与时间步均已收敛”改成与精度相符的表述，例如“ $\Delta t=1\text{ s}$ 的时间离散误差约 $2\times 10^{-3}\text{ }^{\circ}\text{C}$ ，约为附件 1 噪声 ($0.02\text{ }^{\circ}\text{C}$) 的十分之一，结果按 3 位有效数字解读”，并把表 1 / 表 2 的位数与之一致；(b) 若要保留“四位小数已收敛”的说法，就把 Δt 压到 0.01 s 量级重出 result1.xlsx 与两表。

同一件事在问题二上更明显（用包内求解器直接验证）： $N=200$ 、 $\Delta t=5\text{ s}$ 与 $\Delta t=0.25\text{ s}$ 相比，表 3 在 $t=0.5\text{ h}$ 一整行差 $4.4\text{e-}3 \sim 7.0\text{e-}3\text{ }^{\circ}\text{C}$ ——动的是第三位小数 ($32.1966 \rightarrow 32.1896$ 、 $35.4176 \rightarrow 35.4133$)， $t=2\text{ h}$ 差 $2.5\text{e-}3$ 、 $t=3\text{ h}$ 收到 $3.7\text{e-}4$ ；表 4 在 $t=0.5\text{ h}$ 差 $4.3\text{e-}4$ ，末位同样不稳。也就是说表 3 / 表 4 的末位（表 3 甚至是第三位）是时间离散误差，而不只是“第 4 位没意义”。

要彻底解决就得动时间格式：把后向 Euler 换成 Crank - Nicolson ($\theta=1/2$) 后，同样 $N=400$ 、 $\Delta t=1\text{ s}$ 的温度误差可从 $1.9\text{e-}3$ 降到 $1\text{e-}6$ 量级（这是外部复核者给出的数，我没有独立重跑 CN，方向与量级合理）。但换格式会让四个 result 文件、全部表格、插图与对照实验的末位一起变，必须整套重

算并重新验证——对这个已经“最后一轮没同步”的包来说风险大于收益，除非时间充裕，建议走“改表述 + 统一位数”，把 CN 作为论文里的一条改进方向写明。

4.7 【中】随包的“基准值”JSON 里表 6 是旧值

代码/export_baseline_tables.py 的作用是把论文表 1-表 6 导出成输出/基准值-论文表1到表6.json，作为自检比对的备份基准。本次用包内 tex 重跑该脚本（未加参数时它会读论文/论文全文.tex），得到的表 6 与随包文件不同：随包是 1.4770 / 1.2770 / 0.7859 ... / 0.2765，重算出来是 1.7188 / 1.5371 / 1.0222 ... / 0.4205，后者与论文表 6、与 result4.xlsx 一致，即随包那份基准值的表 6 停留在更早一轮的问题四结果上。

目前它还没有造成误判：verify_paper_numbers.py 在包内存在 论文/论文全文.tex 时优先读正文，只有在没有 tex 时才回落到这份 JSON。但一旦在没有 tex 的目录（或把 论文/ 删掉）里自检，表 6 就会拿旧值去比，产生一堆假报警。

建议：重跑 代码/export_baseline_tables.py --tex 论文/论文全文.tex 覆盖该文件；同时该脚本的说明里“本包不包含论文”一句与事实不符（包内有论文/论文全文.tex），建议一并更正。

4.8 【低】文档对 --with-result2 的体积判断与实测不符（实测更有利）

交付说明第四节、README 与 打包支撑材料.sh 的注释都写：加 --with-result2 后约 38 MB、会超过 20 MB 限制，需要与赛区确认。实测不是这样：

项目	文档说法	实测
--with-result2 包体积	约 38 MB，超过 20 MB	16.4 MB（16,445,596 B），脚本自判“符合 ≤20 MB”
包内 result2.xlsx	27 MB，无法再压到 20 MB 以内	27.34 MB → 在外层 ZIP 里压到 10.59 MB，解出后 MD5 与源文件相同

原因：result2.xlsx 虽然是压缩格式，但外层再压一遍仍有约 2.6 倍空间（实测 zlib level 9 可把 27.34 MB 压到 10.59 MB），所以“xlsx 已是压缩格式、无法再压”的推断不成立。结论：完全可以按单一支撑材料包提交，把 result2.xlsx 一并发进去（若仍希望减小体积，再按文档里“只交 small 包 + 由 reproduce.sh --fast 重算 result2”的方案处理）。

4.9 【低】文档细节问题

位置	问题	建议
数据核对报告.md（复核方式一节）	写死了本机绝对路径 /Users/sqz/.cache/codex-runtimes/.../python3，该文件在支撑材料内，等于把本机用户名路径一起交付	改为 ./venv/bin/python
交付说明.md 格式自查表	写“附录……须含……全部源程序 □”，但论文附录只完整列出 11 个程序，另有 8 个（make_figures.py、xlsx_style.py、build_result*.py、check_env.py、export_baseline_tables.py、make_delivery_pdf.py）未列入（论文中已声明“随支撑材料提供”）	或补齐附录，或把自查表表述改为“核心源程序 + 全部源程序随支撑材料提供”
交付说明.md 格式自查表	写“参考文献 1 页（第 31 页）”，实际第 31 页同时是附录 A 的起始页	改为“参考文献起自第 31 页，附录紧随其后”
README.md 第六节	写“本包也不含论文”，但第三节目录结构里列出了论文/论文全文.pdf，容易误解	明确写成“支撑材料包内不含已编译的 PDF”
setup.sh / matplotlib	在 HOME 缓存不可写时会打印 pip 缓存与 matplotlib 配置目录的告警	非错误，实测不影响安装与出图；可在 README 里加一句说明
README.md（2 处）、代码/代码结构.md（3 处）	仍写“数据自检 347 项”，而包内脚本实际审计 365 项	改成 365 项（或改成“约 365 项”以免以后再对不上）

位置	问题	建议
代码/closure_rhod.py 的文档字符串	写“论文表 11 列了三种口径 …… 51.39 h”，但三种口径那张表在正文是表 8，而 51.39 已被更正为 48.39 h；该脚本被 \lstinputlisting 内联进附录，所以交付版 PDF 第 57、76 页就印着这句过期的话	把表号改成表 8，并把第三口径的数字改成 48.39 h（或删掉具体数字）
代码/prepare_outputs.py（ 第 91 行）	提示文字写“末年半径”，应为“末态半径”	改错别字

五、本次无法验证或受环境限制的项

1. 未在 Windows / Linux 上解压验证：本次只在 macOS 上测；zip 的 UTF-8 文件名标志位正确，理论上不会乱码，但没有实测验证。
2. 未在 Python 3.12 上验证：本机只有 3.9.6；README 声称 3.9.6 与 3.12.14 均已通过。
3. 论文 PDF 的重新编译依赖 LaTeX 引擎，包内不含。本次用环境中自带的 Tectonic 与本机包缓存编译成功（83 页）；在没有 TeX 缓存、又完全离线的机器上无法重建 PDF，但这不影响随包交付版的正确性（已逐页比对一致）。
4. 插图的像素级外观受 matplotlib 版本影响：本次用锁定版本 3.9.4，与随包插图存在字体渲染层面的细微差别（文字内容一致），属于已知的正常现象。
5. 交付版的页边距、页码等结论来自对 PDF 的独立测量（逐页量墨迹包围盒、抽取页脚文字），未使用打印机实测。

六、复现步骤（建议按此顺序）

1. 解压 支撑材料.zip 到一个空目录；
2. ./setup.sh 建环境（需联网；离线机器可用 README 第六节给出的 wheel 方案）；
3. ./reproduce.sh --fast 复核四个 result*.xlsx 与正文表格（约 9 分钟）；
4. ./reproduce.sh 复核全部对照、派生数据与插图（约 60 分钟）；
5. ./venv/bin/python 代码/verify_paper_numbers.py --json 输出/数据自检报告.json（需先完成第 3/4 步，生成 *_data.npz）；
6. python3 代码/make_delivery_pdf.py 论文/论文全文.pdf 论文/论文全文_交付版.pdf（需先自行编译 tex 并安装 PyMuPDF）；
7. ./打包支撑材料.sh（或 --with-result2，实测 16.4 MB）重新打包。

七、建议的最小处理清单

优先级	处理	涉及文件
高	重跑 shrink_criterion.py 并让论文 § 11 / 表 20 与该 JSON 一致（或补上能产生论文值的实现与参数）	输出/收缩判据与代价.json、论文/论文全文.tex
高	重跑自检并覆盖旧报告（先解决上一条的 10 项不一致）	输出/数据自检报告.json
中	重跑 make_figures.py，覆盖随包的 7 张过期插图（尤其 s07 的 50.31 h）	输出/figs/*.pdf
中	重跑 compute_shrinkage.py 并同步“路径”一节的 8 个数字	补充分析/斜率截距与收缩/…tex、README.md
中	重跑 export_baseline_tables.py --tex 论文/论文全文.tex，覆盖表 6 仍是旧值的基准值文件	输出/基准值-论文表1到表6.json

优先级	处理	涉及文件
中	问题一把 Δt 改成 0.25 s 重出 result1.xlsx 并同步表 1 / 表 2；或把“时间步已收敛”改为“收敛到 3 位有效数字”并把表收紧到 3 位	代码/solve_p1.py 的默认参数、输出/result1.xlsx、论文表 1 / 表 2
中	在 README 里说明自检前需先跑 solve_p1.py / run_all.py 生成 *_data.npz；requirements.txt 建议补上 pymupdf 并注明还需 LaTeX	README.md、requirements.txt
低	更正“--with-result2 约 38 MB、超限”的说法：实测 16.4 MB，可把 result2.xlsx 一并放进单包	README.md、交付说明.md、打包支撑材料.sh
低	把写死的本机路径改成 ./venv/bin/python	数据核对报告.md
低	统一附录程序清单与自查表、第 31 页归属、README 关于论文 PDF 的表述	交付说明.md、README.md、论文/论文全文.tex
—	处理完成后重新打包并重命名提交（校内编号），保留两个文件的 MD5	支撑材料.zip、论文全文_交付版.pdf

说明：以上全部为测试发现，测试过程中没有修改交付包内的任何代码或数据；所有复现与比对都在临时目录中完成。

附录 A 关键数字对照（论文 / 随包结果 vs 本次复算）

项目	论文 / 随包值	本次复算
问题一 1800 s 中心含水率 / (kg/kg)	2.5500	2.5500
问题一 1800 s 表面含水率 / (kg/kg)	1.5103	1.5103
问题一 1800 s 中心温度 / °C	33.5765	33.5765
问题一 1800 s 表面温度 / °C	36.7863	36.7863
问题一 水分守恒相对残差	6.8e-14	6.847e-14
问题二 3 h 中心含水率 / (kg/kg)	1.7663	1.7663
问题二 3 h 表面含水率 / (kg/kg)	1.0082	1.0082
问题三 烘干时间	206700 s = 57.42 h	206700 s = 57.42 h
问题四 烘干时间（随包 result4.xlsx 末行）	183840 s	183840 s
问题四 烘干时间（判据满足时刻）	183885 s = 51.08 h	183885 s = 51.08 h
网格收敛 N=20 / 50 / 100 / 200 / 400	203335 / 205625 / 206365 / 206700 / 206840 s	完全一致
三种口径（N=100）Fick / 变密度 / 干基	206365 / 219340 / 174195 s	完全一致
§ 11 收缩判据（表 11、表 20）	29.45 / 23.85 / 21.27 h 等	不一致，见 4.1

附录 B 全量复现时间轴（./reproduce.sh）

步骤	内容	累计用时
0/6	环境自检（依赖 + 中文字体）	约 20 秒
1/6	问题一求解（N=400、dt=1 s）与 result1.xlsx、正文表 1/表 2	27 秒
2/6	问题二/三/四求解（N=200、dt=5 s）、result2/3/4.xlsx、正文表 3-表 6	7 分 12 秒
3/6	20 组网格与口径对照、三种口径对照、灵敏度、收缩判据	46 分 09 秒
4/6	插图派生数据（问题一细网格解、不收缩对照、阈值扫描等）	56 分 57 秒
5/6	40 张插图（矢量 PDF + 300 dpi PNG + 灰度预览 + 版面自检）	58 分 24 秒

步骤	内容	累计用时
6/6	数据自检（365 项，10 项不一致，见 4.1）	59 分 48 秒